本章路線

這章要回答的核心問題是：多個執行流如何共享 process 資源，同時維持排程與資料正確性？

前置知識。 process 位址空間、CPU 排程基本概念、stack 與暫存器。

學習目標

- 區分 process 與 thread 的資源邊界
- 比較 threads 映射模型與並行來源
- 辨識 cancellation、signal、TLS 與 fork 的執行緒問題

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

執行緒私有狀態與行程共享資源

執行緒（thread）各自擁有程式計數器（PC）、暫存器、堆疊、執行狀態與身分。同一行程（process）中的執行緒通常共享程式碼、全域變數、heap、位址空間與開啟檔案等資源。共享方便合作，但錯誤指標也可能破壞其他執行緒使用的資料。

各執行緒通常使用不同的堆疊區域，但這是使用方式與物件生命週期的區別，並不是同一位址空間內的硬體隔離：若將指向尚未結束生命週期的堆疊物件的指標交給另一執行緒，後者可能存取該物件。

兩個 worker 的資料	W1	W2	協調方式
暫存器／區域索引	自己的索引 0	自己的索引 0	執行上下文獨立
暫存緩衝區	B1 暫存 110	B2 暫存 120	各自寫入不同區域
共用工作佇列	取得工作 11	取得工作 12	所有取出操作遵守同一同步協定
全域總和	合併 110	合併 120	使用原子操作或 mutex；也可在 join 後由一方合併

執行緒可能提高回應性、降低部分資源建立成本，並利用多核心；實際效果仍取決於工作分解及硬體。增加執行緒不會自動消除鎖競爭、記憶體頻寬限制或序列相依。

全域變數、執行緒區域儲存與生命週期

假設兩條執行緒各執行一次 `counter++`。若它們未同步地修改同一個非原子 C/C++ 物件，衝突存取會構成 data race，行為未定義。以抽象的讀取、加一、寫回模型說明：兩者都讀到 0，再各自寫回 1，便可能遺失一次更新；這只是機制示例，不能宣稱是未定義行為唯一的輸出。

若改用每條執行緒獨有的 TLS（thread-local storage）計數器，兩者各自從 0 增至 1，消除了對同一計數器的共享寫入。要得到總和 2，worker 必須在結束前將各自的值複製到生命週期足夠長、互不衝突的共享結果槽；主執行緒成功 join 兩者後，再讀取結果槽並相加。執行緒結束後不可直接讀取已銷毀的 TLS 物件。

TLS 可跨函式呼叫保存，直到該執行緒結束，與單次函式呼叫的自動區域變數不同。若 thread pool 重用 worker，TLS 不會在每個工作開始時自動歸零；應在工作邊界明確初始化及清理。同步也無法使已失效的堆疊物件重新有效。

並行、平行與工作分解

Concurrency（並行處理）表示多個工作流可以交錯前進；即使只有一顆 CPU，也能透過切換工作達成。Parallelism（平行執行）表示工作在同一時間執行，需要足夠的硬體執行資源。SMT 的多個邏輯執行環境會共享部分核心資源，因此每個邏輯 CPU 不等於一顆完整獨立核心的算力。

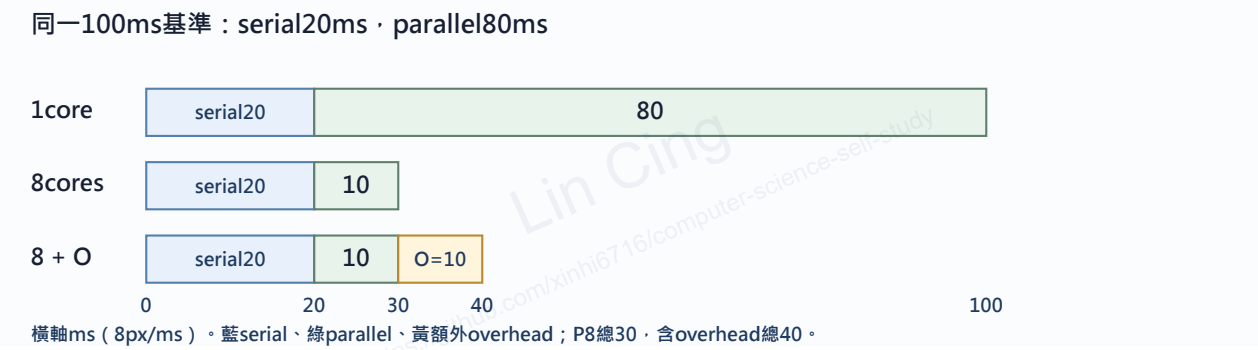
資料平行將相同操作分配到不同資料區段；任務平行則讓不同功能或階段並行。設有 64 MiB 的 32-bit 整數陣列，共 16,777,216 個元素；分成 8 份後，每份 8 MiB、2,097,152 個元素。每個 worker 使用足以容納該輸入總和的 64-bit 累加器先計算區段和，再將 8 個部分和以 7 次加法合併。此做法仍須讀取整個陣列。

若每個元素都以 atomic add 更新同一全域計數器，會有 16,777,216 次共享 read-modify-write；分區版本則在完成時計算並發布 8 個部分和。這是操作數量比較，不是實測加速倍數：記憶體頻寬、快取與 NUMA 配置仍可能成為限制。

Amdahl 定律：以基準時間推算上限

固定相同問題規模：單核心基準時間 $T_1 = 100$ ms，其中 20 ms 必須序列執行，80 ms 可理想地平均分給 P 個核心，且先不計額外開銷。令序列部分占基準時間的比例 $s = 0.2$，則 $T_p = 100[s + (1-s)/P]$ ms，加速比 $S_p = 1/[s + (1-s)/P]$。

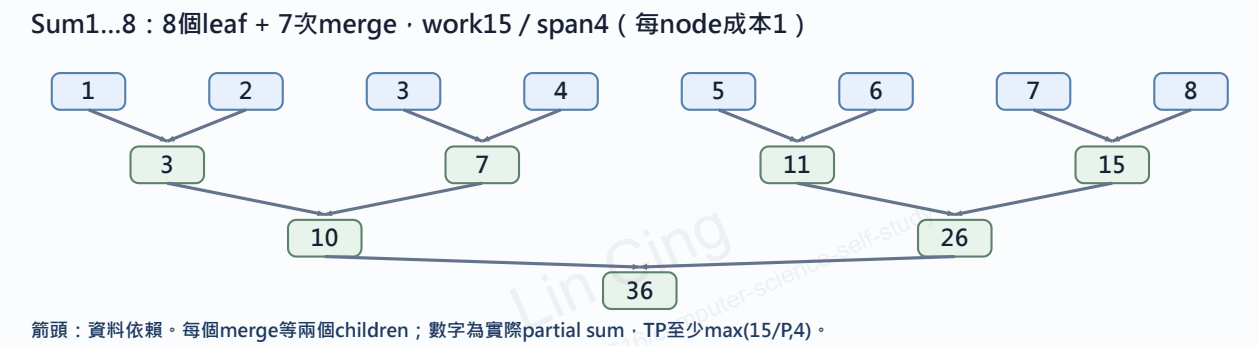
P＝8 時，$T_8 = 20 + 80/8 = 30$ ms，加速比為 100/30＝10/3，效率為 (10/3)/8＝5/12；當 P 趨近無限大，加速比的上限為 1/0.2＝5。



若 8 核心總共多出 10 ms 的啟動與同步成本，$T_8 = 40$ ms，加速比降為 2.5。操作數比例也不能直接當時間比例：80 個操作各需 1 cycle，另 20 個各需 10 cycles，基準時間是 280 cycles；只將前者加速 8 倍後為 10＋200＝210 cycles，加速比僅 4/3。

工作量、關鍵路徑與歸約

將數值 1 至 8 分別放在 8 個 leaf，以二元加法合併；假設每次讀取 leaf 與每次 merge 的抽象成本均為 1 單位。總工作量 work＝8＋7＝15；最長依賴路徑依序經過 leaf、pair、quartet、root，故 span＝4。



第一層得到 3、7、11、15；第二層得到 10、26；最後得到 36，共進行 7 次加法。每個節點都必須等待前驅完成，例如 root 不可能在兩個 quartet 完成前執行。

對任意 P 處理器排程，$T_p \geq \max(15/P, 4)$，因此加速比不超過 work/span＝15/4。這個比值是平均平行度；它不代表超過 3.75 個處理器後就一定沒有額外收益。若某一工作區段特別大，總時間也至少要等最慢的區段完成，不能只看平均值。

False sharing 與 data race 的差別

假設 cache line 為 64 bytes、整數為 4 bytes，且陣列起點按 64 bytes 對齊。W1 反覆寫入 `a[0]`、W2 寫入 `a[1]`：兩者是不同整數物件，在沒有其他衝突存取時可以不存在語言層級的 data race；但它們位於同一 cache line，可能使快取一致性協定反覆轉移該 line 的寫入權，降低效能。

將兩個寫入位置分別放在符合對齊要求、間隔至少 64 bytes 的區域，可降低此模型中的干擾；是否真的改善、改善多少，仍須在目標系統實測。若原本兩者修改的是同一個計數器，padding 不會修復 data race，仍需同步或重新設計資料所有權。

為更新加 mutex 可能提供互斥，但也可能增加序列化；改用每執行緒的計數器則改變資料表示，仍需定義結果如何合併。正確性、cache line 共享成本與結果合併應分別分析。

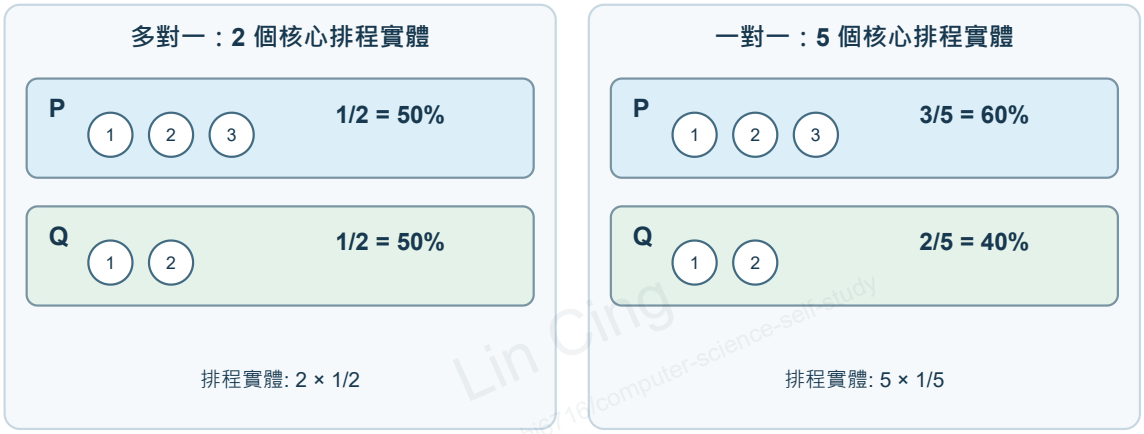
使用者執行緒與核心排程實體的映射

模型	使用者執行緒與核心排程實體的關係	一個真正阻塞的 system call	平行執行限制
Many-to-one	多條使用者執行緒共用一個 carrier	若沒有非同步替代，carrier 阻塞後，同一行程的其他使用者執行緒無法另由核心排程	同時最多一個 carrier 執行
One-to-one	每條使用者執行緒對應一個核心實體	一條阻塞，其他就緒實體仍可排程	受就緒實體數和核心數限制
Many-to-many	runtime 將多條使用者執行緒排到多個 carrier	影響取決於 carrier 數量與阻塞補償策略	受可執行 carrier 數和核心數限制
Two-level	M:N，且允許部分執行緒綁定 carrier	綁定與未綁定執行緒依各自規則運作	綁定會限制 runtime 的排程選擇

使用者程式呼叫哪個 thread library API，不能單獨決定其唯一映射；大量使用者執行緒也不必然代表同樣數量的核心物件。若 runtime 以 nonblocking I/O 和 event loop 等待完成，許多任務可以在邏輯上等待，而 carrier 本身不必阻塞；這與真正阻塞的 system call 不同。

排程單位會影響行程的 CPU 份額

圓圈：使用者執行緒；核心實體長期等份額



假設單 CPU、所有執行緒持續可執行、等優先權，且排程器長期平均分配 CPU 給每個可執行的核心排程實體。P 有 3 條使用者執行緒，Q 有 2 條。

在此例的 many-to-one 模型中，每個行程各有一個 carrier，共兩個核心實體：P、Q 分別取得 1/2 的 CPU。改為 one-to-one，五條執行緒各有一個核心實體、各取得 1/5：P 合計 3/5，Q 合計 2/5。

換一種公平規則，答案也會改變

仍假設單 CPU、P 的三條和 Q 的兩條執行緒持續可執行。若排程器先讓 P、Q 兩個行程群組各得 1/2，再於各群組內平均分配：

分配方式	P 的總份額	P 每條	Q 的總份額	Q 每條
五條核心實體各得 1/5	3/5	1/5	2/5	1/5
先對兩個行程群組平分	1/2	1/6	1/2	1/4

例如觀察時間為 60 ms，群組公平模型給 P、Q 各 30 ms；再平均分配時，P 每條得到 10 ms，Q 每條得到 15 ms。這是模型計算，沒有計入切換成本。

不同權重、執行緒阻塞或多核心也可能改變結果。Many-to-one 模型下，使用者層 runtime 如何分配 P 的 1/2，還需另行約定；不能僅由核心份額推出。

100 條執行緒與 10 個 carrier：阻塞與補償

假設有固定的 10 個 carrier K1 至 K10，不提供替代 carrier，也不使用非同步 I/O；100 個使用者執行緒各想發出一次會阻塞的 I/O。U1 使 K1 阻塞後，還有 9 個 carrier 可執行；當 K1 至 K10 都已阻塞，便沒有 carrier 可執行。此時其餘 90 條使用者執行緒仍在 runtime 的 ready queue，尚未有機會發出它們的 system call。



「最多 10 個核心實體同時阻塞」只來自上述固定 10 個 carrier 的前提，不能推成所有 M:N 系統的通則。若另一策略允許最多 20 個 carrier，原本 10 個阻塞後，K11 至 K20 仍可執行並發出更多阻塞呼叫。若採非同步 I/O，100 個任務也可能都在邏輯上等待完成，而 carrier 不一定阻塞。

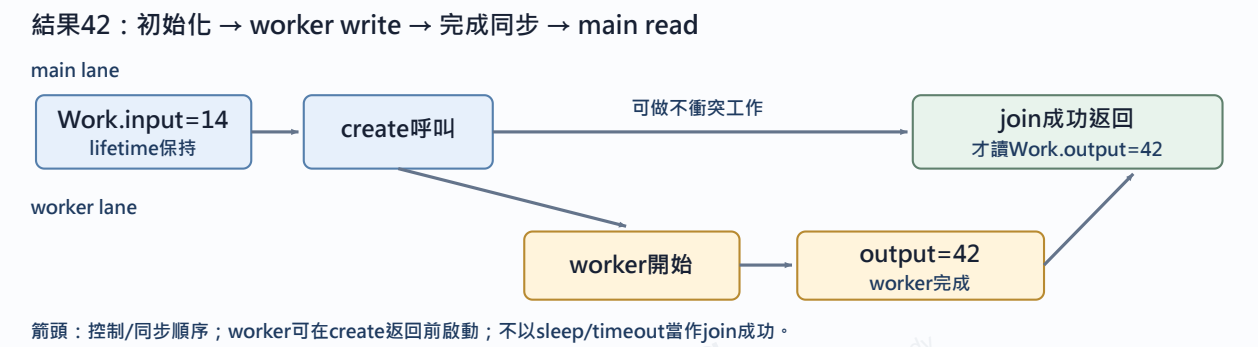
Scheduler activations／upcalls 是協調核心資源通知與使用者層排程的歷史機制：核心通知阻塞或可用資源，runtime 再選擇執行緒。它必須處理 runtime 鎖、通知上下文及 carrier 配置，並不會創造額外 CPU 算力；two-level 綁定也不表示同一條執行緒可同時在兩個 carrier 執行。

建立、完成與 join：結果的生命週期

假設主執行緒持有 `Work`，輸入為 14；worker 計算 `output=3×14=42`。主執行緒在建立 worker 前初始化輸入，並讓 `Work` 至少存活到成功 join 後。建立後不能假設 worker 尚未開始，也不能在缺乏同步時讀取 worker 正在寫入的輸出。

階段	主執行緒	worker	必須維持的條件
建立前	初始化 input＝14	尚未建立	參數與物件有效
建立成功	可做其他不衝突的工作	可開始並寫 output＝42	執行先後不固定
完成	可等待 join	不再使用 Work	完成事件依 API 契約同步
join 成功後	讀 output＝42，之後才結束 Work 的生命週期	已完成	不與 worker 寫入重疊

建立、完成與 join：結果的生命週期 （續）



睡眠 10 ms 不能證明 worker 已完成。Detached thread 不再透過 join 收取，但仍需要另一個完成通知與所有權協定；不能 detach 後立刻釋放它仍可能使用的參數。

Pthreads、Windows 與 Java 的建立和等待

Pthreads 的 `pthread_create` 接收入口函式及參數；成功 `pthread_join` 會等待可 join 的目標結束並取得其結果。應處理函式回傳的錯誤碼，不能一律只查 `errno`。同一目標的 join 所有權必須明定，也不能回傳指向已失效 stack 區域變數的指標。

Windows 可用 `CreateThread` 建立執行緒；只有 `WaitForSingleObject` 對 thread handle 成功取得 signaled 結果，才按該完成契約讀取結果。Timeout 與 `WAIT_FAILED` 不是完成證明。使用完仍須 `CloseHandle`；關閉 handle 本身不會終止執行緒。

Java 8 的 `Thread.start` 啟動新執行緒，直接呼叫 `run` 則是目前執行緒的普通方法呼叫。啟動前的動作 happens-before 新執行緒的動作；worker 的動作 happens-before 另一執行緒成功從 `join` 返回。`InterruptedException` 不代表目標已完成。

執行緒池與 fork-join 的等待風險

假設 8 個工作各需 1 個 CPU 時間單位，4 個 worker、FIFO 排程且沒有額外成本：J1–J4 在 [0,1) 執行，J5–J8 在 [1,2) 執行，完成時間為 2。Pool 可重用 worker；worker 數和等待佇列容量是不同限制，仍需處理 backpressure、取消與錯誤。

Fork-join 將問題拆成子工作再合併。若任務太細，建立、佇列與同步成本可能超過有用工作；任務的合併也必須等待其前驅完成。

例如只有兩個 worker 的 pool 同時執行 A、B；兩者各自提交 A1、B1 到同一個 pool，再阻塞等待子工作。A1、B1 已 ready，卻沒有空閒 worker，因而產生 starvation deadlock，即使資料依賴圖本身沒有環。具備明確 help-join 行為的 runtime 可讓等待者協助執行子工作，但不能假定所有 pool 都有此功能；改用 continuation 也是一種解法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OpenMP reduction 與 GCD 佇列

OpenMP 的 `parallel for` 可分配迭代；若每次迭代寫入不同的 `c[i]`，可分別計算 `c[i]=a[i]+b[i]`。相反地，多個執行緒同時執行 `sum+=a[i]` 需要 reduction 或其他同步。

假設實際建立四個 worker，分別處理 [1,2]、[3,4]、[5,6]、[7,8]，得到部分和 3、7、11、15；一種合法合併順序為 10＋26＝36。OpenMP 不保證這組分配或合併順序；浮點加法並非精確結合，改變次序可能改變最後位元。

GCD 將工作提交到 dispatch queue。Serial queue 在同一時間只執行該 queue 的一個工作；concurrent queue 可重疊，先入列不代表先完成。若正在 serial Q 執行的工作同步提交另一工作到同一 Q 並等待，兩者互相等待，可能卡死。不同 queue 之間的共享物件不會自動互斥。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多執行緒 fork 與 signal 傳遞

假設同一行程中 T2 持有 mutex M，而 T1 呼叫 POSIX `fork`。子行程只保留呼叫者 T1；複製的記憶體中 M 卻可能仍呈現由 T2 持有，而子行程沒有 T2 能釋放它。子行程在成功 `exec` 前須遵守 async-signal-safe 的限制；`pthread_atfork` 只有配合整體鎖與函式庫設計才能處理已知問題。

Signal disposition 通常由行程中的執行緒共享，signal mask 則是每條執行緒各自維護。同步 fault 對應造成事件的執行緒；process-directed signal 依遮罩與系統規則交給符合條件的執行緒，不是每條執行緒都執行一次 handler。使用專門的 `sigwait` 執行緒時，其他執行緒也須協調遮罩。

成功 `exec` 會替換程式映像，原本其他執行緒不繼續執行舊映像；建立一般執行緒則會共享現有行程資源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延後取消與安全的清理時機

`pthread_cancel` 成功只表示取消請求已提出，不等於目標已終止。Deferred cancellation 在指定的 cancellation point 處理；asynchronous cancellation 可能在更新資料或持鎖期間發生，因此更難維持不變量。

假設 worker 擁有 mutex M 和 buffer B，並在 `pthread_cond_wait` 等待穩定的條件。先停用取消，建立資源及清理程序，依序登記 unlock-M、free-B；再啟用 deferred cancellation。若等待期間處理取消，`pthread_cond_wait` 先重新取得 M，清理程序再依後進先出執行：先釋放 B，後解鎖 M。主執行緒成功 join 後才能回收需要等待 worker 完成的共享參數。

清理程序只處理資源，不會自動修復資料結構的半完成更新；應在不變量穩定處設取消點，或設計可驗證的 rollback。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取消前後的資源表與不變量

階段	mutex M	buffer B	釋放 B 次數	解鎖 M 次數
資源與清理程序已建立	worker 持有	有效	0	0
condition wait 期間	原子釋放 M 並等待	有效	0	0
開始處理取消	依等待契約重新取得 M	有效	0	0
第一個 cleanup	worker 仍持有 M	已釋放	1	0
第二個 cleanup	已解鎖	已釋放	1	1

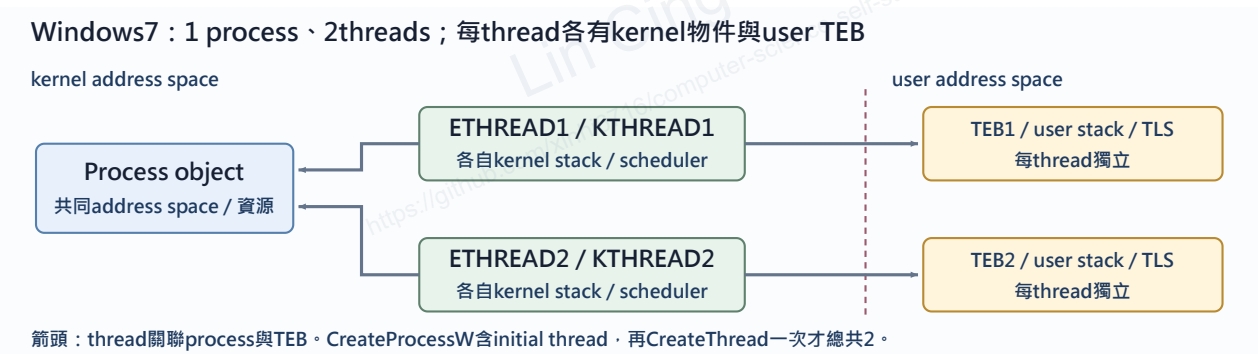
正常、錯誤和取消路徑都要配對清理並明定所有權，避免重複釋放或解鎖。不能一邊留下半更新的共享資料，一邊只靠 unlock 宣稱恢復正確；必要時需延後取消或回復狀態。

Windows 7 與 Linux 3.2 的執行緒物件

以原書的 Windows 7 及 Linux 3.2 歷史案例為例，比較行程擁有的資源與核心實際排程的單位；本頁不是對現代版本實作細節的通用保證。

Windows 的 process 保存位址空間與行程層級資源；thread 分別有核心中的 ETHREAD、KTHREAD，以及使用者空間的 TEB。`CreateProcessW` 已建立一個初始 thread；若接著成功呼叫一次 `CreateThread`，這個案例共有兩個 thread，而非一個。

Linux 3.2 用 `task_struct` 表示每個可排程 task。一般 `fork` 建立新的 task 與 kernel stack，且未共享 `CLONE_VM` 時建立不同的 mm 和頁表；`clone` 則可透過 flags 選擇共享 mm、檔案表與 signal handler 等結構。



Windows 7 與 Linux 3.2 的執行緒物件 （續）

比較項目	Windows 7	Linux 3.2
可排程單位	kernel thread	task
個別狀態	暫存器、堆疊、TEB	暫存器、kernel stack、排程狀態
共享範圍	同一 process 的位址空間及資源	由 clone flags 決定 mm、files、fs 和 signal handlers

在指定的雙執行流例子中，Windows 有 1 個 process、2 個 thread 與 2 個 TEB。Linux 若用一般 fork，則有 2 個 task、2 個 mm；若以題設的 thread-group clone 共享 mm，則有 2 個 task、1 個 mm，但各自有 kernel stack。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立呼叫後的狀態與版本反例

Windows 案例中，每個 thread 有獨立的暫存器、user／kernel stack 與 TEB，但引用同一個 process 和位址空間。第一條 thread 已由 `CreateProcessW` 建立，新增一條後總數才是 2。

Linux 的一般 `fork` 建立 child 自己的 task、kernel stack、mm 與頁表；某些私有頁面可先透過 COW 共享實體 frame，這不表示兩者共享 task 物件。對仍共享的 COW 頁面寫入時，核心處理寫入保護 fault，必要時才建立私有實體副本；若符合重用條件，不一定發生複製。

指定的 thread-group clone 包含 `CLONE_THREAD`：兩個 task 的 TID 不同、TGID 相同。Linux 3.2 中 `CLONE_THREAD` 要求 `CLONE_SIGHAND`，而後者要求 `CLONE_VM`。即使只使用 VM／FS／FILES／SIGHAND 共享資源，缺少 `CLONE_THREAD` 也不能推論兩者屬於同一 thread group。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立呼叫後的狀態與版本反例 （續）



共用位址空間不代表衝突的 heap 寫入自動安全，仍須同步。成功 exec 改變程式映像；特定版本在不可回復點前的錯誤與之後的失敗，處理方式可能不同，不能保證所有失敗都能繼續執行原映像。

從結果、完成契約與依賴評估執行緒

計算 1 至 8 的和可以由單一執行緒完成，也可以分成四份或使用 fork-join；答案都應是 36。平行版本還須確認部分結果存活、依完成契約同步，並在前驅完成後才合併。建立執行緒本身不能保證結果正確。

固定工作量的 Amdahl 模型使用基準時間比例；work／span 下界來自工作總量與依賴路徑；carrier 數則限制可供核心排程的實體。這三者描述不同限制，不能由其中之一推論「執行緒數等於核心數必然最佳」或「超過平均平行度就一定沒有收益」。

實作改變哪一項，便重新檢查哪個前提

變更	需要重新檢查的條件
從 1:1 改成 M:N	阻塞時 carrier 是否補償，以及 runtime 的排程規則
從各自暫存區改成共享資料	衝突存取、同步與資料所有權
從 join 改成 detach	完成通知與參數、結果的生命週期
多執行緒行程呼叫 fork	子行程保留哪些執行緒，以及遺留鎖的處理
加入 cancellation	正常、錯誤與取消路徑的資源清理和資料不變量

比較系統效能時，應固定輸入及正確結果、可用核心與記憶體配置，明定計時範圍，再量測吞吐量與延遲分布。平均吞吐量可能掩蓋鎖競爭或工作池耗盡造成的長尾，也應記錄失敗與等待情況。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study